

Chapter - 1

Introduction

Text Books:

1. Operating System Concepts 10e, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne.
2. Operating Systems Design and Implementation, Third Edition, Tanenbaum.

Internal Evaluation:

Internal Exams I = 10

Internal Exams II = 10

Assignment = 10 [* copied assignment = 0]

Lab Reports + class performance + Quizz = 5 + 5 + 5

Group Presentation 4/5 people = 5

Introduction to Operating system

- An operating system is a program that acts as an intermediary between a user of a computer and the computer hardware.
- The purpose of an operating system is to provide an environment in which a user can execute programs.
- Acts as interface between application and hardware



Abstract view of components of Operating system

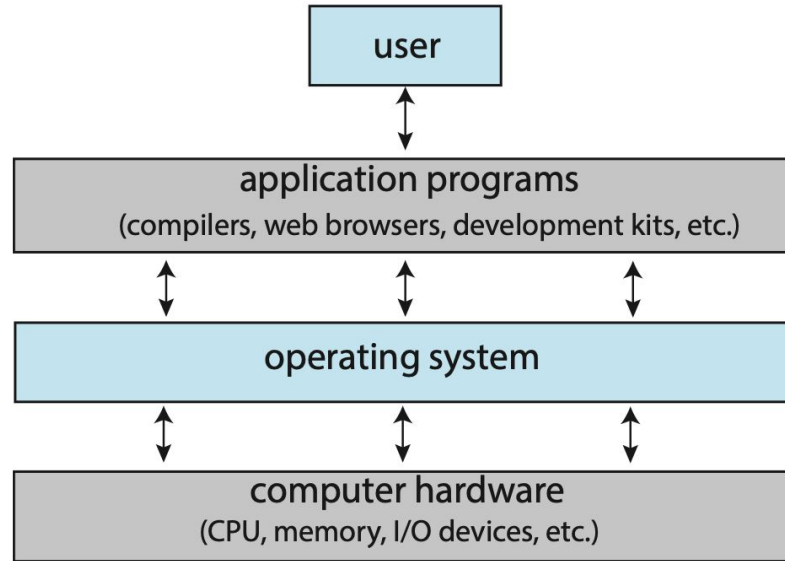


Figure 1.1 Abstract view of the components of a computer system.

Types of Operating System

- Serial Processing
- Simple Batch
- Multiprogramming
- Time Sharing systems (Multi-Tasking)
- Multi- Processing
- Parallel System
- Distributed Systems
- Real time Systems

Serial Processing

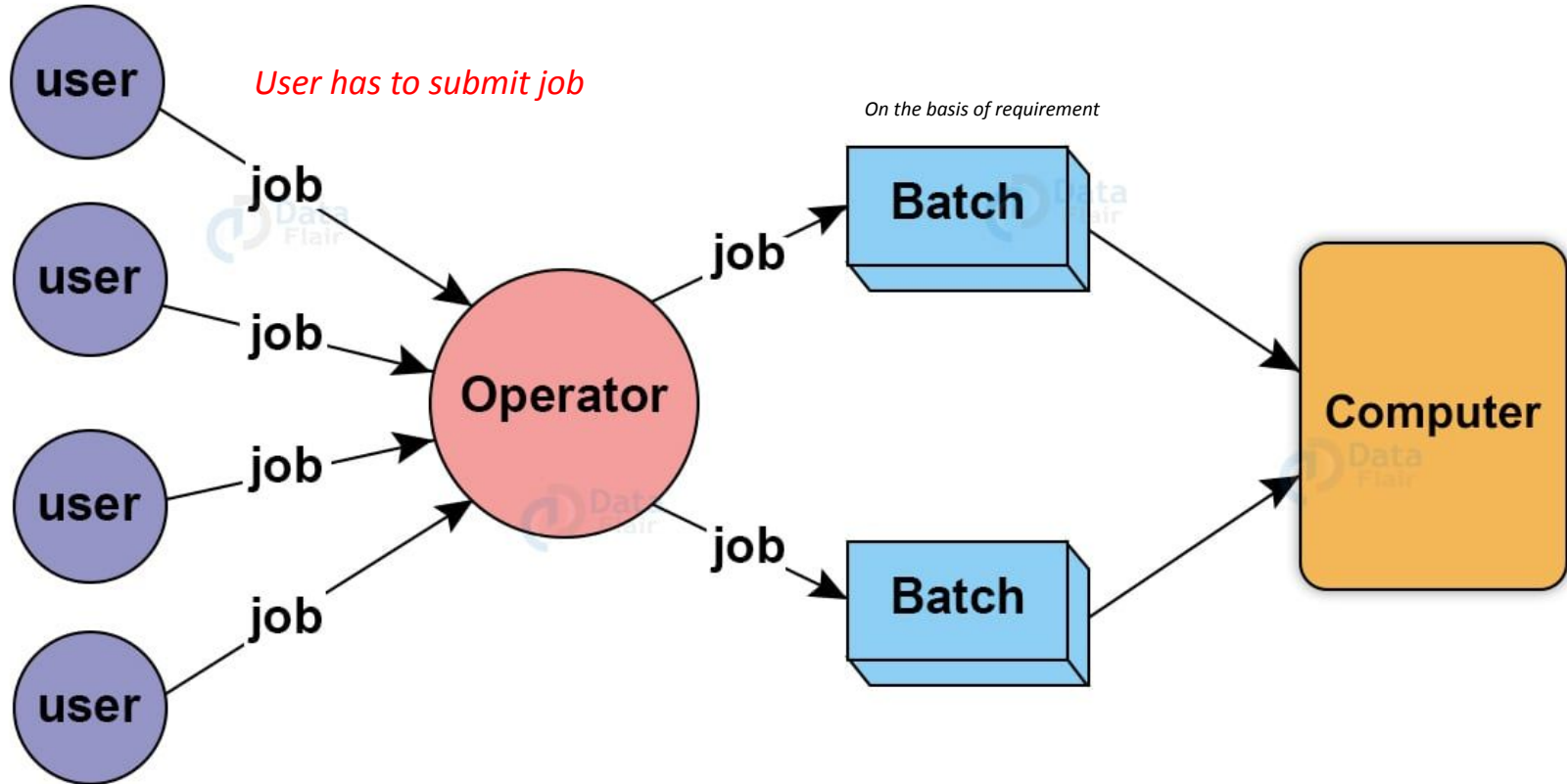
- At **early stage** the programmer **interacted directly with the computer hardware**, because there was **no operating system**.
- Jobs are executed one after another, in the exact order they arrive **FIFO**.
- User have access to the computer in series.
- If an error halted the program, the error condition was indicated by the **lights** and programmer would **proceed to examine register and main memory** to determine the cause of the error.
- If the program proceeded to a normal completion, the output appeared on the printer. (Results were printed directly on paper using a line printer.No screen or monitor existed)

Simple Batch System

Batch : set of jobs with similar needs

- In the **batch** system, it requires the grouping of similar jobs which consists of programs data and system commands.
- Jobs with similar needs are batched together and run as a group.
- There is no direct interaction b/w user and computer.
- Thus, the programmers left their programs with the operator.
- The operator then sorts programs into batches with similar requirements.

Batch Operating System



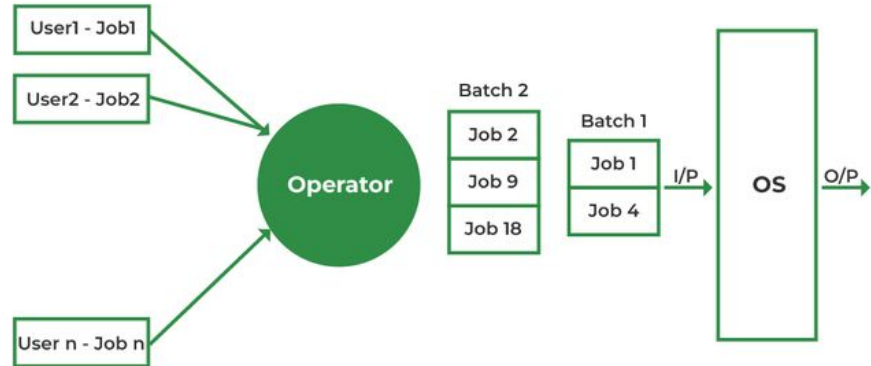
- **Simple Batch System**

- Since the jobs are executed in the FIFO manner, the batch OS require very simple CPU scheduling techniques.
- In addition, batch system allows only one user program to reside in the memory at a time, thus memory management is also very simple affairs in batch OS.
- Since only one program is in execution at any time, critical device management is not required, which simplifies the I/O management.
- Since the files are accessed in a serial manner, a concurrency control mechanism is not required, making the file management also very simple matter in batch OS.

• Simple Batch System

• The problems with Batch Systems are as follows:

- Lack of interaction between the **user and the job**.
- CPU is often **idle**, because the speed of the mechanical I/O devices is slower than the CPU.
- Difficult to provide the desired **priority**.



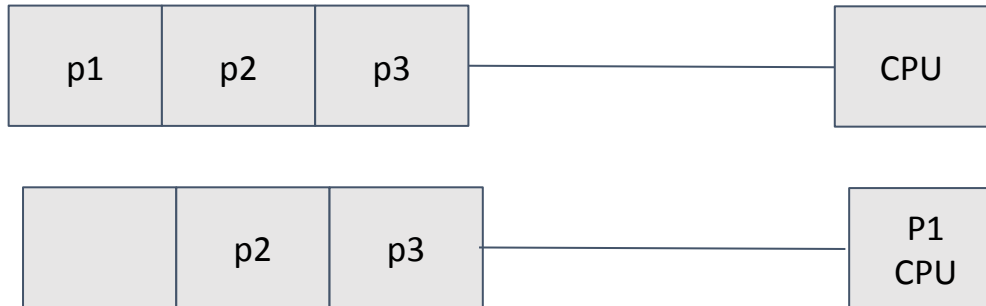
Multiprogramming system

- A Multiprogramming Batch System is where multiple **jobs are loaded in memory**, and the CPU switches between them whenever one job waits for I/O, keeping the **CPU always busy**.

1. Multiple jobs in memory simultaneously

Example:

Job A, Job B, Job C loaded together.



Multiprogramming system

- The idea here is :
 - The operating system keeps several jobs in **memory** at a time.
 - This set of jobs is a subset of the jobs kept in the job pool.
 - The operating system picks and begins to execute one of the jobs in memory.
 - Eventually, the job may have to wait for some task, such as a tape to be mounted, or an i/o operation to complete.
- In a multi-programming system, the operating system simply switches to and executes another job and so on.
 - Eventually , the first job finishes waiting and gets the CPU back.
- As long as there is always some job to execute, the CPU will never be idle.
- If several jobs are ready to be brought into memory, and there is not enough room for all of them, the system must choose among them, called job scheduling.
- Having several programs in memory at the same time requires having some form of memory management.
- If several jobs are ready to run at the same time, the system must choose among them.

Time-sharing (Multi- tasking)

- Time-sharing or multitasking is a **logical extension of multiprogramming**.
- Processor's time which is shared among multiple users simultaneously is termed as time-sharing.

Multiprogrammed Batch Systems	Time-Sharing Systems
• maximize processor use	• minimize response time.

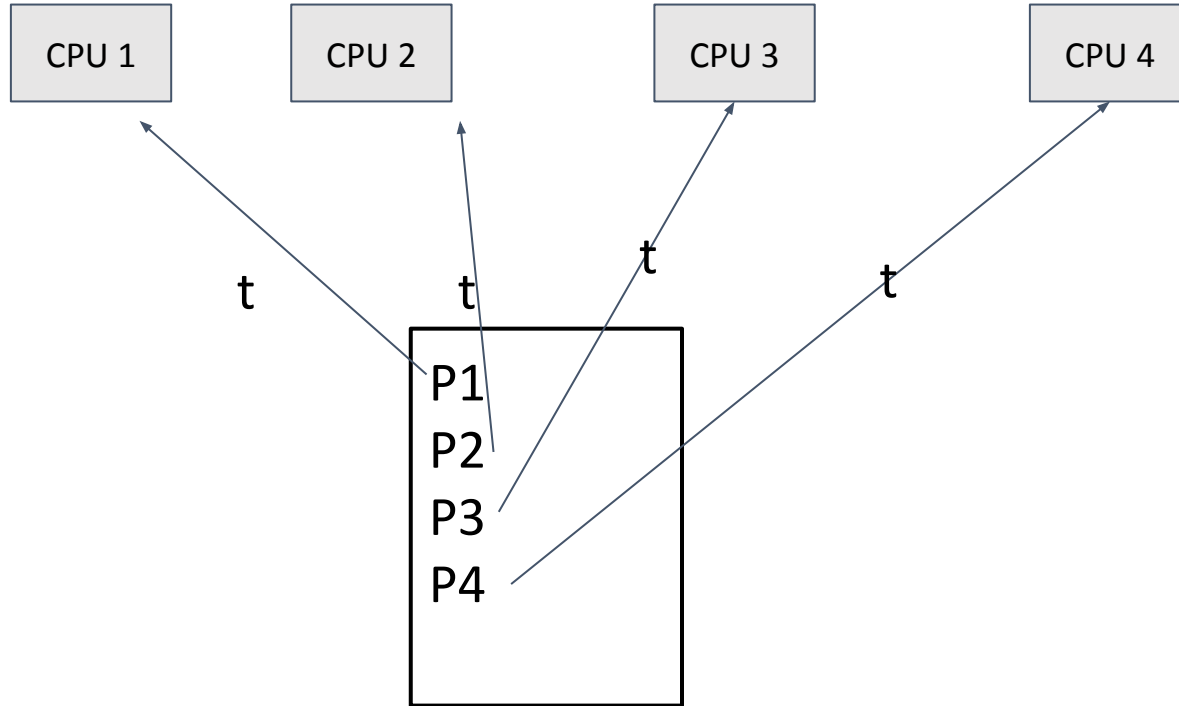
- Multiple jobs are executed by the CPU by switching between them, but the switches occur so frequently. Thus, the user can receive an immediate response.

A **Time-Sharing OS** allows multiple users to use a computer at the same time by rapidly switching the CPU among them, giving each user a tiny time slice.

For eg: A time slice is a small amount of time (like 10 ms) that the CPU gives to each user or program to run.

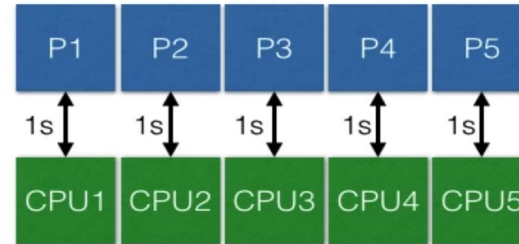


Multi-Processing



Parallel Processing:

- Technique is designed to **speed up the execution process** of programs by dividing them into **multiple segments** and processing them simultaneously through shared memory.
- Such systems are called **tightly coupled systems**.



Several reasons for building such systems:

- **Increased throughput:** more CPUs more numbers of works done
- **Less costly:** Save money compared to multiple single system since multi-processor system share the resource of a single computer.
- **Increased reliability:** since work is distributed properly among processors, failure of one processor will not halt the system, remaining processor must pick share of work of failed processor.
 - This ability to continue providing service proportional to the level of surviving hardware is called “graceful degradation”.
 - Systems that are designed for graceful degradation are also called fault-tolerant.

*Note: Speed up execution by dividing work.

Parallel system

- There are two types of multi-processing system:
 - **Symmetric-multiprocessing model:** each processor runs identical copy of the operating system and these copy communicate with one another as needed.
 - **Asymmetric multi-processing:** each processor is assigned a specific task. A master processor controls the system; the other processors either look to the master for instruction or have pre-defined task. This is called **Master-Slave** relationship. The master processor schedules and allocates work to slave processors.
- Many popular operating systems including, Windows, Unix and Linux, run on multiprocessors.

Quick look:

Multi- processing	Many Program running at once at same time t.
Parallel Processor	Single Program being broken down and executed simultaneously.

Distributed system

- In contrast to the tightly coupled systems, the processors do not share memory or a clock.
- Instead each processor has its own local memory.
- The processors communicate with one another through various communication lines, such as high-speed buses or telephone lines.
- These systems are usually referred to as **loosely coupled**, or distributed systems.
- The processors in distributed system may vary in size and function.
- They may include small microprocessors, workstations, minicomputers, and large general-purpose computer system.

Distributed system

- The major reasons of building distributed systems are:

1. Resource sharing: If a number of different sites (computers) with different capabilities are connected to one-another, then a user at one site may be able to use resources available at another.

- Eg. a user at one site A may be using the laser printer at another site B.
- Provides mechanism for sharing files at remote sites, processing information in a distributed database.

2. Computation speed:

- If a particular computation can be partitioned into a number of sub-computations that can run concurrently, then a distributed system may allow us to distribute the computation among various sites to run that computation concurrently.
- Load sharing: sharing between different sites.
 - When a web service receives many requests, the OS can use a load-sharing approach to distribute the incoming http requests across multiple web servers.

Distributed system

- The major reasons of building distributed systems are:

3. Reliability: If one site fails in a distributed system, the remaining sites can potentially continue operating.

4. Communication: There are many instances in which programs need to exchange data with one another on one system. Users may initiate file transfers or communicate with one another via electronic mail. A user can send mail to another user at the same site or at a different site.

Personal computer

- Since the cost of hardware have decreased, it has become feasible to have a computer system dedicated to a single user.
 - This type of computers are referred to as “personal computers (PCs)”.
 - Single user system ,--> portable
- Modern ones all support multiprogramming, often with dozens of programs started up at boot time to provide support to a single user.
- Examples include Windows, Linux, Macintosh OS etc.

Real-time system

- A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment.
- It is a multitasking operating system that aims at executing real-time applications.
- Real-time operating systems often use specialized scheduling algorithms so that they can achieve a deterministic nature of behavior.
- The main object of real-time operating systems is their quick and predictable response to events.
- They either have an event-driven design or a time-sharing one.
- An event-driven system switches between tasks based on their priorities while time-sharing operating systems switch tasks based on clock interrupts.

Real-time system

- **Hard real-time systems** guarantee that critical tasks complete on time. In hard real-time systems, secondary storage is limited or missing and the data is stored in ROM. In these systems, virtual memory is almost never found.
- **Soft real-time systems** are less restrictive. A critical real-time task gets priority over other tasks and retains the priority until it completes. Soft real-time systems have limited utility than hard real-time systems. For example, multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers, etc.